Your Steps: Make It Live & Finish Up

Do these in order. Steps 1–5 take about 30 minutes total and only need a web browser (plus one short terminal command in Step 2). Step 6 is optional and lets me finish the database items myself.

⏱ **Steps 1–5 ≈ 30 minutes**

---

1 Change your database password — DO TODAY

Your old password was visible in the code. Until you change it, anyone who saw the code can open your database.

1. Open supabase.com/dashboard → your project → **Settings** → **Database**.
2. Click **Reset database password**. Copy the new password somewhere safe.
3. Copy the two new connection strings (the **pooled** one ending in `:6543`, and the **direct** one ending in `:5432`).
4. In **Vercel** → **your project** → **Settings** → **Environment Variables**, update:
   - `DATABASE_URL` = the **pooled** string (add `?pgbouncer=true&connection_limit=1` at the end if it isn't there).
   - `DIRECT_URL` = the **direct** string.
5. In **GitHub** → **repo** → **Settings** → **Secrets and variables** → **Actions**, update any database secret used by the automated jobs.
6. (Optional) In Supabase → **Logs** → **Postgres**, glance for any connection you don't recognise.

☐ Done — password changed and updated everywhere

2 Apply the database update — ~5 min

I prepared the database changes safely (nothing was touched live). You apply them once. They only **add** things — no data is deleted.

In a terminal on the project (with `DIRECT_URL` pointed at your new direct connection), run:

```
npx prisma migrate resolve --applied 00000000000000_baseline
npx prisma migrate deploy
npx prisma migrate status
```

The last command should say **"Database schema is up to date."** Run it during a quiet period (it briefly builds an index). Not comfortable in a terminal? Use Step 6 and I'll do it for you.

- [ ] Done — migration applied, status is "up to date"

3 Point Vercel at the safe build command ~2 min

The old build command could delete live data on every deploy. The new one can't.

1. **Vercel → Settings → General → Build & Development Settings**.
2. Set **Build Command** to:

```
npm run build
```

Do **not** add the migration command here — you ran it in Step 2 so that preview deployments never touch your live database.

- [ ] Done — build command set to `npm run build`

4 Cancel Apify ~2 min

Your code doesn't use Apify at all. If you're paying for it, it's wasted money.

Log into your Apify account and cancel the subscription (or confirm you don't have one). Nothing in the app depends on it.

- [ ] Done — Apify cancelled / confirmed not subscribed

5

Set the new environment variables ~8 min

A few security fixes now **require** their own secret (instead of insecurely reusing another). The app will refuse to run insecurely without them.

In **Vercel → Settings → Environment Variables** (and Railway for the Python service), add any that are missing. To generate a strong random value, run `openssl rand -hex 32` in a terminal.

Name	Where	Required?	What it's for
`JWT_REFRESH_SECRET`	Vercel	**Yes**	Mobile login security. `openssl rand -hex 32`
`CREDENTIAL_ENCRYPTION_KEY`	Vercel	**Yes**	Encrypts stored integration keys. `openssl rand -hex 32`
`AI_API_SECRET`	Railway + Vercel	**Yes**	Locks down the Python AI service.
`PREVIEW_ADMIN_PASSWORD`	Vercel (preview)	Optional	Leave unset to keep the preview-login disabled (recommended).
`UPSTASH_REDIS_REST_URL` + `_TOKEN`	Vercel	Optional	Better rate-limiting/caching at scale. Falls back safely if unset. Free at upstash.com.
`GITHUB_TOKEN`	Vercel	Optional	Raises the admin-dashboard data limit. Falls back safely if unset.

Also: in Supabase's table editor, **delete the row** `preview@scamdunk.com` from the `AdminUser` table if it exists (it was created with the old default password).

- [ ] Done — required secrets set; preview admin row removed

6 Optional: give me database access so I can finish the rest

This is your Question 1. Below is the safe way to do it — and an honest note on what it does and doesn't unblock.

Strong recommendation: do NOT give me your live production database. Instead, give me a throwaway **branch database** — a real, separate copy I can apply migrations to and test against, with zero risk to your live data.

1. **Create a branch database.** In Supabase, use **Database Branching** (or just spin up a second free Supabase project as a sandbox). This gives you a separate connection string.
2. **Give the session that connection string.** In Claude Code on the web, open your **environment settings** and add it as a secret/environment variable named `DATABASE_URL` (and `DIRECT_URL`). The how-to is here: code.claude.com/docs/en/claude-code-on-the-web (see "environments / environment variables").
3. **Allow the connection.** That same settings area controls the **network policy** — make sure outbound access to your Supabase host is permitted, or the session can't reach it.
4. **Tell me it's ready.** Use the button below; I'll apply the migration, verify it, and start the database-dependent items.

> **Honest note on what DB access unblocks:** it lets me **apply & verify the migration** and **build/test the admin-dashboard rebuild**. It does **not** finish the two data-dependent items — tuning the score thresholds and re-training the AI — because those need your *labelled history* (which past stocks were real scams), not just a database connection. If that labelled data lives in your database or files, share where, and I can use it.

Click to copy an instruction, then paste it into Claude Code:

☐ (Optional) Done — access provided / decided to skip

When Steps 1–5 are done, deploy the `claude/compassionate-clarke-s6xqk1` branch to a Vercel **preview** first, click around, then merge to `main`. Stuck on anything? Paste your question into Claude Code — everything is saved in GitHub, so it's safe to experiment.